ICS-344: Information Security

DVSA Vulnerability Discovery and Remediation

Course Project Report

Course / Section:	ICS-344 / Term 252
Student Name(s) + ID(s):	Team 17. Replace with the actual student names and IDs before final graded submission.
Your DVSA Website URL:	https://<redacted-dvsa-site-url>
AWS Region:	us-east-1
Date:	5 May 2026
Vulnerability Title(s):	Lessons 1 through 10 from the official DVSA project brief

This repository version keeps secrets, account identifiers, JWTs, and private URLs redacted. The public report is structured to match the assignment template while preserving the course requirement that DVSA be used only in a legal non-production lab environment.

Contents

Project Context

This report follows the project requirements in `project_description/Project Description.pdf`. The scope is the 10 official DVSA lessons. For each lesson, the report presents the intended workflow, the vulnerability, the exploit path, the impact, the remediation, and the post-fix verification. The supporting repository artifacts are organized under `vulnerabilities/`, `fixes/`, `screenshots/`, `diagrams/`, `scripts/`, `report/`, `presentation/`, and `videos/`.

The DVSA lab environment represented in this repository is an AWS serverless application using Amazon S3, API Gateway, AWS Lambda, DynamoDB, SQS, SES, IAM, CloudWatch, and CloudTrail. All testing was intended for a controlled educational environment only.

Deliverables Check

Required deliverable	Repository location	Status
Written report PDF	`report/final-report.pdf`; source in `report/main.tex`	Present
Presentation slides	`presentation/slides.pdf`; source in `presentation/project-slides.tex`	Present
GitHub repository evidence	`vulnerabilities/`, `fixes/`, `screenshots/`, `diagrams/`, `scripts/`	Present
Demo video recording	Google Drive link in `videos/README.md`	Linked

1 Lesson 1: Event Injection

Part 1) Goal and Vulnerability Summary

This lesson demonstrates event injection in the DVSA order API path. The affected component is the API Gateway to Lambda request-processing workflow. A crafted payload can make backend code treat attacker-controlled content as executable behavior instead of plain data, which can lead to code execution in the lab Lambda context.

Part 2) Why This Works / Root Cause

The root cause is unsafe deserialization or dynamic handling of user-controlled input. If the handler or one of its dependencies accepts serialized functions or evaluates request data as code, the Lambda runtime can execute unintended logic before normal validation completes.

Part 3) Environment and Setup

- Region: `us-east-1`
- Entry point: `/dvsa/order`
- Backend path: order-processing Lambda workflow
- Evidence sources: `vulnerabilities/lesson-01-event-injection/report-source.pdf`, CloudWatch log proof, and `redacted-requests.txt`

Part 4) Reproduction Steps

1. Locate the API Gateway invoke URL used by the DVSA frontend.
2. Append the order path and send a crafted JSON payload containing the event-injection marker.
3. Observe the client response, which may be a generic internal error.
4. Open CloudWatch logs for the order Lambda.
5. Confirm that backend-side execution evidence appears in the logs.

Part 5) Evidence and Proof

- `vulnerabilities/lesson-01-event-injection/report-source.pdf` contains the imported evidence package.
- `vulnerabilities/lesson-01-event-injection/redacted-requests.txt` stores the sanitized payload shape.
- `screenshots/lesson-01/` stores endpoint discovery, exploit request, normal response, exploit evidence, and post-fix request screenshots.
- CloudWatch is the primary proof source because API Gateway may hide backend execution behind a generic error.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the request parsing path. Remove unsafe deserialization, parse requests as strict JSON, and validate all fields against an allowlisted schema. User input must never be evaluated as code.

Part 7) Code / Config Changes

- `fixes/lesson-01/before-code.txt`: vulnerable request-handling pattern
- `fixes/lesson-01/after-code.txt`: safe parsing and validation pattern
- `fixes/lesson-01/before-example.txt` and `after-example.txt`: behavior comparison
- `fixes/lesson-01/explanation.md`: remediation explanation

Part 8) Verification After Fix

Repeat the crafted payload. The fixed backend should reject it as invalid input, and CloudWatch should no longer show injected code execution. Normal order handling should still work with valid data.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 01: Event Injection	Request data must remain data and must never execute inside the Lambda handler.	API payload, CloudWatch logs, source report PDF, and screenshot evidence.	Normal order requests follow the expected handler path and complete without injected behavior.	The crafted event causes backend execution evidence in CloudWatch.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 01: Event Injection	The backend executed behavior derived from request data, which violates the intended separation between data and code.	Intentional misuse / security-relevant abuse	Safe JSON parsing and schema validation in the order request path.	The crafted payload is rejected and no backend execution evidence remains.

Part 10) Takeaway / Lessons Learned

Serverless functions still execute ordinary application code with cloud permissions. Unsafe parsing can turn a simple API request into backend code execution, so input must be treated strictly as data.

2 Lesson 2: Broken Authentication

Part 1) Goal and Vulnerability Summary

This lesson demonstrates broken JWT authentication in the DVSA order workflow. The backend can trust a forged or modified token if it uses decoded claims without complete verification. The impact is user impersonation and unauthorized access to another user's order data.

Part 2) Why This Works / Root Cause

JWT claims are visible to the client and can be edited. The weakness occurs when the backend uses identity fields such as `username` or `sub` without verifying the token signature, issuer, audience, expiry, and trusted subject.

Part 3) Environment and Setup

- Region: `us-east-1`
- Endpoint: DVSA order API
- Users: one attacker account and one victim account in the lab
- Tools: browser DevTools, decoded JWT inspection, and direct API testing
- Evidence source: `vulnerabilities/lesson-02-broken-authentication/report-source.docx`

Part 4) Reproduction Steps

1. Create or identify two lab users and place at least one order for each.
2. Capture a valid JWT from the attacker account through DevTools.
3. Inspect or decode the token to identify the identity claims.
4. Modify the forged token payload to use the victim identity claim.
5. Send the forged request to the order API.
6. Observe unauthorized access to the victim user's order data.

Part 5) Evidence and Proof

- `screenshots/lesson-02/from-report/` contains 8 extracted screenshots from the source report.
- `vulnerabilities/lesson-02-broken-authentication/redacted-requests.txt` stores the sanitized request pattern.
- The proof condition is that a forged token returns data belonging to another user.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the backend order-authentication path. The server must verify token integrity and trusted claims before using identity. Invalid signatures, wrong issuer or audience, expired tokens, and inconsistent subject claims must be rejected.

Part 7) Code / Config Changes

- `fixes/lesson-02/before-order-manager.js`: vulnerable pattern that trusts decoded claims
- `fixes/lesson-02/after-order-manager.js`: verified JWT pattern using trusted claims
- `fixes/lesson-02/explanation.md`: remediation explanation

Part 8) Verification After Fix

Repeat the forged-token request. It should be rejected, while a valid user token should still return only that user's own orders.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 02: Broken Authentication	Only verified identity claims may authorize order access.	Browser workflow, JWT payload inspection, API request and response evidence, and source-report screenshots.	A valid user token returns only that user's own order data.	A forged token returns another user's order data.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 02: Broken Authentication	Caller-controlled claims were treated as proof of identity even though they were not cryptographically verified.	Intentional misuse / security-relevant abuse	JWT signature and claim verification in the backend order-access path.	Forged tokens are rejected and valid tokens still return only the caller's own data.

Part 10) Takeaway / Lessons Learned

Decoding a JWT is not authentication. The backend must verify the token before trusting any claim.

3 Lesson 3: Sensitive Data Exposure

Part 1) Goal and Vulnerability Summary

This lesson shows that DVSA receipt or storage workflows can expose private data through API responses, generated files, logs, or object access. The main impact is disclosure of user receipt or order-related information outside the intended ownership boundary.

Part 2) Why This Works / Root Cause

The root cause is weak authorization and weak data minimization around receipt-related data. If the backend exposes receipt content or access links without checking ownership, users can access information outside their intended boundary.

Part 3) Environment and Setup

- Region: `us-east-1`
- Services: API Gateway, Lambda, DynamoDB, S3 receipts, and CloudWatch
- Evidence source: `vulnerabilities/lesson-03-sensitive-data-exposure/report-source.pdf`
- Supporting artifacts: redacted request samples and screenshot evidence

Part 4) Reproduction Steps

1. Create normal receipt or order data in the lab.
2. Trigger the vulnerable receipt path or API response path.
3. Capture the exposed response, file, or object evidence.
4. Confirm that the accessed data should belong only to the owner.
5. Redact identifiers before storing the proof.

Part 5) Evidence and Proof

- `screenshots/lesson-03/` contains endpoint discovery, token capture, DynamoDB evidence, sensitive file evidence, exploit evidence, and post-fix results.
- `vulnerabilities/lesson-03-sensitive-data-exposure/redacted-requests.txt` stores the sanitized request or response sample.
- The exploit is proven when receipt data or receipt access becomes available without proper ownership checks.

Part 6) Fix Strategy / Probable Mitigation

Add ownership checks before returning receipt data or signed URLs. Keep receipt objects private, generate short-lived access only after authorization, and avoid returning unnecessary sensitive fields.

Part 7) Code / Config Changes

- `fixes/lesson-03/before-code.txt` and `after-code.txt`: pre-fix and post-fix behavior
- `fixes/lesson-03/before-example.txt` and `after-example.txt`: evidence-oriented comparison
- `fixes/lesson-03/explanation.md`: remediation explanation

Part 8) Verification After Fix

Repeat the receipt access attempt as an unauthorized user. The response should be denied or sanitized, while the legitimate owner can still access the correct receipt data.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 03: Sensitive Data Exposure	Users may access only their own receipt data and only the minimum required fields should leave the backend.	API responses, receipt objects, source report PDF, screenshots, and redacted request patterns.	Authorized owner access returns the expected receipt information only.	Private receipt data or links become visible outside the ownership boundary.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 03: Sensitive Data Exposure	Private data crossed the authorization boundary because the backend exposed more than it should.	Intentional misuse / security-relevant abuse	Ownership checks and private receipt access controls in the receipt retrieval path.	Unauthorized access is denied or sanitized, while owner access still works.

Part 10) Takeaway / Lessons Learned

Sensitive data controls must be enforced where data leaves the backend, not only where it is stored.

4 Lesson 4: Insecure Cloud Configuration

Part 1) Goal and Vulnerability Summary

This lesson demonstrates that insecure S3 configuration can allow untrusted files into the DVSA receipt-processing workflow. The affected component is the receipts bucket and its trust relationship with downstream Lambda processing. The impact is that a storage misconfiguration becomes a backend attack path.

Part 2) Why This Works / Root Cause

The root cause is weak S3 public-access or bucket-policy configuration combined with Lambda processing that trusts bucket contents. If unauthorized users can place objects where Lambda expects application-generated receipts, the backend processes untrusted input.

Part 3) Environment and Setup

- Region: `us-east-1`
- AWS services: S3 bucket, Lambda receipt-processing path, AWS Console or CLI
- Evidence source: `vulnerabilities/lesson-04-insecure-cloud-configuration/report-source.`
- Supporting artifact: `redacted-requests.txt`

Part 4) Reproduction Steps

1. Inspect the receipt bucket permissions and public-access settings.
2. Attempt a controlled upload of a receipt-like object from outside the intended trust boundary.
3. Observe whether backend processing accepts or reacts to the object.
4. Capture S3 and Lambda evidence from the lab environment.

Part 5) Evidence and Proof

- `screenshots/lesson-04/from-report/` contains 12 extracted screenshots from the source report.
- `vulnerabilities/lesson-04-insecure-cloud-configuration/redacted-requests.txt` stores the sanitized CLI pattern.
- The exploit is proven when unauthorized upload or access reaches the trusted receipt-processing path.

Part 6) Fix Strategy / Probable Mitigation

Enable S3 Block Public Access, restrict bucket policy and ACLs, validate object keys and file types, and allow only trusted application roles to write objects that trigger Lambda processing.

Part 7) Code / Config Changes

- `fixes/lesson-04/before-example.txt`: permissive pre-fix state
- `fixes/lesson-04/after-example.txt`: tightened bucket and workflow controls
- `fixes/lesson-04/explanation.md`: remediation explanation

Part 8) Verification After Fix

Repeat unauthorized upload or access attempts. They should fail, while the legitimate receipt workflow continues to work.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 04: Insecure Cloud Configuration	Only trusted roles and workflows may write receipt objects that the backend processes.	S3 configuration evidence, source report screenshots, CLI patterns, and Lambda workflow context.	Legitimate receipt objects are created and processed only by the intended application workflow.	The bucket accepts input or access beyond the intended trust boundary.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 04: Insecure Cloud Configuration	A cloud resource granted more access than the application trust model intended.	Accidental misconfiguration	S3 Block Public Access and restricted bucket policy in the receipt bucket configuration.	Unauthorized write and access attempts fail while legitimate receipt handling remains functional.

Part 10) Takeaway / Lessons Learned

Cloud configuration is part of application security. A storage misconfiguration can become a backend exploit path when downstream services trust stored content.

5 Lesson 5: Broken Access Control

Part 1) Goal and Vulnerability Summary

This lesson demonstrates broken access control in the DVSA administrative order-update workflow. A normal user can influence privileged order state behavior that should be restricted to administrative logic. The impact is unauthorized order state modification.

Part 2) Why This Works / Root Cause

The backend accepts a valid user context but does not enforce the required role or group before sensitive functionality. In addition, public-facing Lambda invocation permissions may allow external paths to reach internal functions that should be more tightly restricted.

Part 3) Environment and Setup

- Region: `us-east-1`
- Components: order API, admin update logic, JWT claims, and Lambda invoke permissions
- Evidence source: `vulnerabilities/lesson-05-broken-access-control/report-source.docx`
- Supporting artifact: `redacted-requests.txt`

Part 4) Reproduction Steps

1. Create an order as a normal user.
2. Capture the user token and the order identifier.
3. Send a crafted update request toward the privileged path.
4. Observe unauthorized order-state modification.

Part 5) Evidence and Proof

- `screenshots/lesson-05/from-report/` contains 3 extracted screenshots from the source report.
- `vulnerabilities/lesson-05-broken-access-control/redacted-requests.txt` stores the sanitized request example.
- The exploit is proven when a non-admin request changes privileged order state.

Part 6) Fix Strategy / Probable Mitigation

Enforce admin role checks server-side before privileged updates. Restrict Lambda invoke permissions so public-facing functions cannot invoke arbitrary internal functions.

Part 7) Code / Config Changes

- `fixes/lesson-05/before-example.txt`: vulnerable access-control behavior

- `fixes/lesson-05/after-example.txt`: corrected authorization and invoke scope
- `fixes/lesson-05/explanation.md`: remediation explanation

Part 8) Verification After Fix

Repeat the normal-user privileged update attempt. It should fail, while legitimate order creation and billing continue to work.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 05: Broken Access Control	Only authorized administrative workflow may update privileged order state.	JWT claims, API request evidence, Lambda behavior, report screenshots, and remediation notes.	A normal user can create an order and use only the intended user workflow.	A normal user reaches privileged update behavior and changes restricted state.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 05: Broken Access Control	The application authenticated the caller but did not authorize the sensitive action correctly.	Intentional misuse / security-relevant abuse	Admin role checks and narrowed Lambda invoke permissions in the admin update path.	Unauthorized update attempts are denied and the normal order workflow still works.

Part 10) Takeaway / Lessons Learned

Authentication proves who the caller is. Authorization still has to decide what that caller may do.

6 Lesson 6: Denial of Service

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a denial-of-service condition in the DVSA billing workflow. The affected path is the `/dvsa/order` billing action, which reaches the `DVSA-ORDER-BILLING` Lambda function. Repeated billing requests can consume backend processing capacity and reduce availability for legitimate users.

Part 2) Why This Works / Root Cause

The vulnerable design does not sufficiently limit repeated or concurrent billing requests at the API edge. The billing path performs backend work for each request and depends on limited Lambda and API Gateway capacity. Without throttling, idempotency, or per-user controls, a small burst of duplicate requests can degrade the backend.

Part 3) Environment and Setup

- Region: `us-east-1`
- API endpoint: `https://<api-id>.execute-api.us-east-1.amazonaws.com/dvsa/order`
- Action: `billing`
- Backend function: `DVSA-ORDER-BILLING`
- Evidence sources: browser workflow, PowerShell or curl output, CloudWatch logs, Lambda metrics, and API Gateway throttling settings

Part 4) Reproduction Steps

1. Confirm the normal billing and order workflow works from the DVSA frontend.
2. Capture the billing request from browser DevTools.
3. Send one baseline billing request with the captured request body and a redacted Authorization header.
4. Run a controlled repeated-request test with a small serialized count.
5. Run a controlled parallel-job test against the same lab endpoint.
6. Compare returned status codes and response bodies.
7. Open CloudWatch and Lambda metrics for `DVSA-ORDER-BILLING` to confirm backend impact.

Part 5) Evidence and Proof

- `screenshots/lesson-06/` contains screenshots for normal workflow, captured request details, repeated-request output, Lambda metrics, CloudWatch logs, and API Gateway throttling.
- The controlled parallel test produced mixed `200`, `500`, and `502` responses.
- Mixed backend failures during a small parallel burst show that duplicate billing requests can degrade availability.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs at both the API Gateway and backend workflow layers. API Gateway should enforce throttling so small bursts do not reach Lambda unchecked. The billing function should also be made idempotent per order and user so duplicate billing attempts are rejected cheaply and consistently.

Part 7) Code / Config Changes

- `fixes/lesson-06/before-example.txt`: pre-fix repeated-request behavior
- `fixes/lesson-06/after-example.txt`: post-fix throttling and idempotency behavior
- `fixes/lesson-06/explanation.md`: mitigation summary

Part 8) Verification After Fix

Repeat the same controlled test after throttling and idempotency controls are applied. Excess requests should be throttled or rejected cleanly instead of causing backend instability. The normal checkout workflow should still work for legitimate users.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 06: Denial of Service	Billing requests should be processed fairly, and duplicate or concurrent bursts should not exhaust backend capacity.	Browser flow, captured API request, PowerShell output, CloudWatch logs, Lambda metrics, and API Gateway throttling settings.	Normal billing workflow screenshots and baseline single-request evidence.	Parallel repeated-request evidence showing mixed `200`, `500`, and `502` results.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification

Lesson 06: Denial of Service	A small burst of concurrent requests caused backend instability instead of controlled throttling or graceful rejection.	Intentional misuse / security-relevant abuse	API Gateway throttling plus backend idempotency or rate-control design for billing.	Excess requests are throttled or rejected cleanly while normal checkout still succeeds.

Part 10) Takeaway / Lessons Learned

Availability is a security property. Critical serverless workflows need throttling, idempotency, and graceful failure under burst conditions.

7 Lesson 7: Over-Privileged Function

Part 1) Goal and Vulnerability Summary

This lesson demonstrates an over-privileged Lambda execution role. The affected function is `DVSA-SEND-RECEIPT-EMAIL`, whose role originally included permissions broader than the receipt-email workflow requires. The impact is increased blast radius because any code running inside the function inherits the role's temporary credentials.

Part 2) Why This Works / Root Cause

Lambda functions run with execution-role credentials provided by AWS. If a function role allows broad access, a compromised or abused function can perform every action allowed by that role, even when those actions are unrelated to the function's intended purpose. The root cause is failure to apply least privilege.

Part 3) Environment and Setup

- Region: `us-east-1`
- Function: `DVSA-SEND-RECEIPT-EMAIL`
- Services involved: Lambda, IAM, SES, S3, DynamoDB, CloudWatch, and IAM Policy Simulator
- Supporting evidence: screenshots and redacted IAM policy examples

Part 4) Reproduction Steps

1. Open AWS Lambda and locate `DVSA-SEND-RECEIPT-EMAIL`.
2. Open the function permissions tab and follow the execution-role link.
3. Review attached policies for broad actions such as `ses:*` or wildcard resources.
4. Open IAM Policy Simulator and test representative SES, S3, and DynamoDB actions.
5. Record which actions are allowed before the fix.
6. Replace the broad policy with a least-privilege policy.
7. Re-run the simulator and confirm unnecessary actions are denied.
8. Run the normal DVSA order workflow to confirm legitimate receipt behavior still works.

Part 5) Evidence and Proof

- `screenshots/lesson-07/` contains Lambda page, execution-role page, broad SES evidence, simulator checks, post-fix simulation, and normal workflow screenshots.
- `diagrams/lesson-07-blast-radius.png` shows the blast-radius analysis.
- `fixes/lesson-07/before-policy.json` and `after-policy.json` provide the policy comparison.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the Lambda execution role. Broad managed policies and wildcard-resource permissions should be removed. The replacement policy should allow only the actions required for the receipt workflow, together with minimum CloudWatch logging permissions.

Part 7) Code / Config Changes

- `fixes/lesson-07/before-policy.json`: broad pre-fix IAM policy
- `fixes/lesson-07/after-policy.json`: least-privilege target policy
- `fixes/lesson-07/explanation.md`: remediation and verification notes

Part 8) Verification After Fix

After applying the least-privilege policy, re-run IAM Policy Simulator checks for unnecessary SES, S3, and DynamoDB actions. Broad or unrelated actions should be denied, and the normal order and receipt workflow should still work.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 07: Over-Privileged Function	The receipt-email function should have only the permissions required to send receipts and write logs.	Lambda permissions page, IAM role policies, IAM Policy Simulator, screenshot evidence, and the blast-radius diagram.	Normal order and receipt workflow still works after the IAM fix.	Broad SES and unrelated service access were allowed before the fix.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 07: Over-Privileged Function	The role allowed actions broader than the function required, increasing blast radius if the function was abused.	Accidental misconfiguration	Least-privilege IAM policy on the `DVSA-SEND-RECEIPT-EMAIL` execution role.	IAM Policy Simulator denies unrelated actions and the normal workflow still works.

Part 10) Takeaway / Lessons Learned

In serverless systems, the function role is a security boundary. Least privilege reduces blast radius so one function issue does not become broad access to unrelated AWS services.

8 Lesson 8: Logic Vulnerabilities

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a business-logic race condition in the DVSA order workflow. Billing and order-update operations can be submitted close together, producing inconsistent order state. The impact is that the final order state can differ from what was billed.

Part 2) Why This Works / Root Cause

The root cause is a non-atomic read, check, and write sequence. Billing trusted order state that could change before the final write completed.

Part 3) Environment and Setup

- Region: `us-east-1`
- Components: `DVSA-ORDER-BILLING`, `DVSA-ORDER-UPDATE`, and DynamoDB order state
- Evidence source: `vulnerabilities/lesson-08-logic-vulnerabilities/report-source.docx`
- Supporting artifact: `redacted-requests.txt`

Part 4) Reproduction Steps

1. Create an order in the DVSA lab.
2. Submit billing and update requests close together.
3. Inspect the charged amount and final order state.
4. Compare the expected consistent state with the actual state that was stored.

Part 5) Evidence and Proof

- `screenshots/lesson-08/from-report/` contains 13 extracted screenshots from the source report.
- `vulnerabilities/lesson-08-logic-vulnerabilities/redacted-requests.txt` stores the sanitized workflow request.
- The exploit is proven when conflicting operations leave the order in a state inconsistent with the intended workflow.

Part 6) Fix Strategy / Probable Mitigation

Use DynamoDB conditional writes or transactions for order state transitions. Lock or reject updates after billing begins so conflicting state transitions cannot both succeed.

Part 7) Code / Config Changes

- `fixes/lesson-08/before-example.txt`: pre-fix state-management behavior
- `fixes/lesson-08/after-example.txt`: post-fix conditional-write behavior
- `fixes/lesson-08/explanation.md`: remediation explanation

Part 8) Verification After Fix

Repeat the timing test. One operation should succeed cleanly, and the conflicting operation should be rejected instead of silently producing inconsistent state.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 08: Logic Vulnerabilities	Billing and order updates must preserve one consistent order state.	Workflow requests, DynamoDB order evidence, source report screenshots, and remediation notes.	Normal workflows produce one consistent billed state for each order.	Near-simultaneous requests leave state inconsistent with the expected billing result.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 08: Logic Vulnerabilities	The workflow trusted state that changed during a race window, so valid requests in the wrong sequence became unsafe.	Intentional misuse / security-relevant abuse	Conditional writes or transactional state checks in the billing and update workflow.	Conflicting operations are rejected and one consistent order state is preserved.

Part 10) Takeaway / Lessons Learned

Valid requests can become unsafe in the wrong sequence. Security-sensitive workflows need atomic state transitions.

9 Lesson 9: Vulnerable Dependencies

Part 1) Goal and Vulnerability Summary

This lesson shows that an unsafe Node.js dependency can create backend execution risk in the DVSA order request path. The impact is that dependency behavior becomes part of the application attack surface and can support code-execution style exploits.

Part 2) Why This Works / Root Cause

The vulnerable package accepts dangerous serialized function markers. Without dependency review, pinning, and scanning, unsafe package behavior becomes part of the deployed application and can enable executable deserialization behavior.

Part 3) Environment and Setup

- Region: `us-east-1`
- Components: Node.js Lambda package or dependency path and order request processing
- Evidence source: `vulnerabilities/lesson-09-vulnerable-dependencies/report-source.pdf`
- Supporting artifact: `redacted-requests.txt`

Part 4) Reproduction Steps

1. Identify the vulnerable dependency used in the request path.
2. Relate the package behavior to the event-injection condition in the lab.
3. Demonstrate the behavior in the controlled environment.
4. Document the remediation path by removing or replacing the dependency.

Part 5) Evidence and Proof

- `screenshots/lesson-09/` contains endpoint discovery, injection request, normal response, exploit evidence, and post-fix request evidence.
- `vulnerabilities/lesson-09-vulnerable-dependencies/report-source.pdf` stores the source evidence package.
- The exploit is proven when dependency behavior supports request-driven backend execution.

Part 6) Fix Strategy / Probable Mitigation

Remove or replace unsafe packages, use safe JSON parsing, pin reviewed versions, and add dependency scanning to the build and deployment process.

Part 7) Code / Config Changes

- `fixes/lesson-09/before-code.txt`: vulnerable dependency-driven parsing pattern
- `fixes/lesson-09/after-code.txt`: safe parsing pattern
- `fixes/lesson-09/before-example.txt`, `after-example.txt`, and `explanation.md`: remediation evidence

Part 8) Verification After Fix

Repeat the injection payload and confirm that the risky dependency behavior no longer causes backend execution. Dependency review or scanning should also show that the unsafe package is absent or remediated.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 09: Vulnerable Dependencies	Dependencies must not evaluate attacker-controlled request data.	Source report PDF, dependency notes, exploit screenshots, and before or after code examples.	Normal request handling treats request data as plain JSON.	A third-party package enables executable deserialization behavior.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 09: Vulnerable Dependencies	The application inherited unsafe behavior from a dependency, so request data could cross into code execution.	Intentional misuse / security-relevant abuse	Dependency removal or replacement in the request-parsing path, plus safer parsing practice.	The payload no longer executes and the dependency risk is removed or reduced.

Part 10) Takeaway / Lessons Learned

Dependencies are part of the attack surface. Secure code can still fail if unsafe packages are deployed with it.

10 Lesson 10: Unhandled Exceptions

Part 1) Goal and Vulnerability Summary

This lesson demonstrates unsafe exception handling in the DVSA order API. Malformed input can trigger backend exceptions that expose internal details to clients. The impact is disclosure of implementation details such as file paths, exception names, or code references.

Part 2) Why This Works / Root Cause

The backend assumes required fields exist and does not consistently catch exceptions at the handler boundary. This can leak diagnostic details to the caller instead of keeping them in internal logs.

Part 3) Environment and Setup

- Region: `us-east-1`
- Components: `/dvsa/order` endpoint, API Gateway, affected Lambda handler, and CloudWatch
- Evidence source: `vulnerabilities/lesson-10-unhandled-exceptions/report-source.docx`
- Supporting artifact: `redacted-requests.txt`

Part 4) Reproduction Steps

1. Capture a normal order request.
2. Remove or alter required fields in a controlled malformed request.
3. Send the malformed request to the order API.
4. Observe the returned error response.
5. Redact any sensitive details before storing the evidence.

Part 5) Evidence and Proof

- `screenshots/lesson-10/from-report/` contains 2 extracted screenshots from the source report.
- `vulnerabilities/lesson-10-unhandled-exceptions/redacted-requests.txt` stores the malformed request shape.
- The exploit is proven when backend diagnostics are visible to the client instead of remaining only in CloudWatch.

Part 6) Fix Strategy / Probable Mitigation

Validate input before business logic and add centralized error handling. Client responses should be generic, while detailed diagnostics remain in CloudWatch.

Part 7) Code / Config Changes

- `fixes/lesson-10/before-example.txt`: unsafe pre-fix error handling
- `fixes/lesson-10/after-example.txt`: controlled post-fix error handling
- `fixes/lesson-10/explanation.md`: remediation explanation

Part 8) Verification After Fix

Repeat the malformed request. The client should receive a generic safe error, while detailed backend diagnostics remain only in internal logs.

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 10: Unhandled Exceptions	Client errors must not expose backend internals.	Malformed request sample, source report screenshots, API error evidence, and CloudWatch logging context.	Normal requests return the intended application response without internal diagnostics.	Malformed input returns backend diagnostic details to the caller.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 10: Unhandled Exceptions	The API exposed implementation details through its public error surface instead of containing them in logs.	Intentional misuse / security-relevant abuse	Input validation and centralized safe error handling in the order handler path.	Malformed requests return generic client-safe errors while detailed diagnostics remain internal.

Part 10) Takeaway / Lessons Learned

Errors are part of the public interface. Backend diagnostics belong in logs, not in client responses.

Final Redaction and Consistency Notes

Text artifacts in the repository were checked for common secret patterns. The intentional placeholder string `AWS_SECRET_ACCESS_KEY=REDACTED` appears in the README as an example only. Screenshots and imported report sources should still be visually reviewed before submission because image-based evidence can contain values that simple text scanning cannot inspect.

This repository version is organized so that a grader can move directly from the written report to the lesson folder, the fix artifacts, and the screenshots for the same lesson number.